

Catalogue of Logging Guidelines for Improving Automated Log Analysis

Dilsan Mahadeva

RUHR-UNIVERSITÄT BOCHUM

Master's Thesis – August 22, 2026
Chair of Software Engineering.

Supervisor: Thorsten Berger
Advisor: Mukelabai Mukelabai
Boris Karl Schlein



Abstract

Software systems generates large amounts of log data for debugging, monitoring, anomaly detection and root cause analysis. A very sufficient log analysis depends of the quality and consistency from these logs. Because logging practices applied inconsistently and unclear guidelines by developers individually results shows poor quality of logging formats, structures and information.

This thesis investigates software logging from both developer and guideline perspectives. First, logging practices and challenges appeared by software developers during software development and maintenance are examined. Second, existing logging guidelines are collected from academic literature and open-source software repositories. The identified guidelines are organized in a structured catalogue and analysed sorted into themes and characteristics, including what to log, where to log, how to log and log-level verbosity. Finally, the identified guidelines are compared with the practices and challenges reported by software developers to compare and cover the practical logging needs with the guidelines from the catalogue.

The thesis provides insight about an overview of existing logging guidance and software development practice. The contribution is by uploading a logging guideline catalogue to refer for software developers.

Official Declaration

Hereby I declare, that I have not submitted this thesis in this or similar form to any other examination at the Ruhr-Universität Bochum or any other institution or university.

I officially ensure, that this paper has been written solely on my own. I hereby officially ensure, that I have not used any other sources but those stated by me. Any and every parts of the text which constitute quotes in original wording or in its essence have been explicitly referred by me by using official marking and proper quotation. This is also valid for used drafts, pictures and similar formats.

I also officially ensure that the printed version as submitted by me fully confirms with my digital version. I agree that the digital version will be used to subject the paper to plagiarism examination.

Not this English translation, but only the official version in German is legally binding

Eidesstattliche Erklärung

Ich erkläre, dass ich keine Arbeit in gleicher oder ähnlicher Fassung bereits für eine andere Prüfung an der Ruhr-Universität Bochum oder einer anderen Hochschule eingereicht habe.

Ich versichere, dass ich diese Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen benutzt habe. Die Stellen, die anderen Quellen dem Wortlaut oder dem Sinn nach entnommen sind, habe ich unter Angabe der Quellen kenntlich gemacht. Dies gilt sinngemäß auch für verwendete Zeichnungen, Skizzen, bildliche Darstellungen und dergleichen.

Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Ich erkläre mich damit einverstanden, dass die digitale Version dieser Arbeit zwecks Plagiatsprüfung verwendet wird.

DATE

DILSAN MAHADEVA

Erklärung

Als öffentlich finanzierte Forscher wollen wir durch die Veröffentlichung dieser Arbeit einen Beitrag zur Forschungsgemeinschaft leisten, damit andere Forscher dieses Wissen nutzen und weiterentwickeln können. Ich bin damit einverstanden, dass meine Dissertation dauerhaft und öffentlich gespeichert wird. Sie kann dann z. B. von Google Scholar indexiert werden, so dass sie von anderen Forschern gefunden und referenziert werden kann.

Consent

As publicly funded researchers, we aim to contribute to the research community by publishing this work, so other researchers can use and further develop this knowledge. I agree that my thesis will be permanently and publicly stored. It can then be indexed by, for instance, Google Scholar so that it can be found and referenced by other researchers.

DATE

DILSAN MAHADEVA

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Goals and Research Questions	3
1.4	Organization of this Thesis	5
2	Background	7
2.1	Software Logging	8
2.2	Logging Guidelines and Practices	8
2.3	Automated Log Analysis	8
2.4	Logging in Open-Source Software	8
2.5	Related Work	8
2.6	Research Gap	9
3	Methodology	11
3.1	Research Design	11
3.2	Developer Study	11
3.3	Literture Guideline Collection	11
3.4	Open-sorce Repository Guideline Collection	11
3.5	Guideline Classification	13
3.6	Comparison and Gap Analysis	13
3.7	Threats to Validity	13
4	Implementation	15
4.1	Duties and Agreements	15
4.1.1	Rules for Students	15
4.1.2	Rules for Supervisors	16
4.2	Hints on Typesetting	16
4.2.1	Structuring Text, English Hints	16
4.2.1.1	Text	16
4.2.1.2	English Hints	17
4.2.1.3	General Hints	18
4.2.2	Formulas, Figures, Tables, Definitions	19
4.2.2.1	Formulas	19
4.2.2.2	Fontawesome	19
4.2.2.3	Figures	19

4.2.2.4	Tables	20
4.2.3	Glossaries	21
4.2.4	Listings	21
4.2.4.1	Listings	21
5	Results	23
6	Conclusion	25
	List of Figures	27
	List of Tables	28
	List of Algorithms	29
	Bibliography	29
A	Java Code	31

1 Introduction

The first chapter introduces the importance of software logging and challenges of inconsistent logging practices and log data. Based on these challenges, the motivation and problem are described for this research. Afterwards, the next chapter describes goals and research questions about logging practices and guidelines. Followed up by the structure of this thesis.

1.1 Motivation

Modern software systems generate large amounts of log data, which provides valuable insights about runtime behaviour. Logs are widely used for debugging, system monitoring, anomaly detection, failure diagnosis and root cause analysis. In large scale and distributed software systems, the amount of generated log data can become very fast too large for efficient manual inspection. As a result, automated log analysis techniques have become very important for supporting developers and operators in analysing system behaviour and identifying relevant events [1] [2]. The effectiveness of automated log analysis does not only depend on the analysis technique. It also depends on the quality and consistency of the log data that is provided as input. The latest studies show that such inconsistencies can reduce accuracy and reliability [3] [4] [5]. Automated techniques generally require log messages to contain useful information and to follow structures that can be processed reliably. Inconsistent message formats, missing contextual information, unsuitable log levels or poorly placed log statements can make the analysis more difficult and may reduce the quality of the generated log data.

This creates an important problem because the quality of logs are already influenced by developers individually. Developers have to decide what information should be logged, where logging statements should be placed, how log messages should be created and which log level should be used [6] [7]. Even in the industry logs are still inconsistent [8]. These decisions are often made during software development and may differ between individual developers, teams, projects, programming languages and application domains. This makes the investigation of logging guidelines an interesting research problem. It is unclear which logging recommendations are suggested, how detailed or specific they are, whether different sources provide similar guidance and which areas of logging receive the most attention. Moreover, it is also relevant to investigate whether guidelines exist for the different decisions developers

face when implementing logging and whether these recommendations could support the generation of logs that are more suitable for automated analysis.

Consequently, even software systems that generate large amounts of log data may not necessarily generate logs that are well suited for later automated analysis. Existing research has investigated different approaches for improving automated log analysis, for example through more effective parsing and analysis techniques. However, improving the analysis method alone does not solve problems that already exist in the input data. If important information is missing or logging statements are implemented inconsistently, an analysis technique can only process the information that has originally been recorded. There are open source projects [9] and even grey literature [10] [11] for logging guidelines and practices, but these are not sufficient practices and guidelines. Therefore, it is also relevant to investigate how log data can be improved at its source by considering the logging practices used during software development.

The motivation for this thesis is additionally supported by observations from industrial software projects, where logging structures and conventions can be inconsistent or only partially documented. Especially in larger projects involving several developers and software components, incomplete or unclear logging guidance can result in different logging styles within the same system. A more structured understanding of existing logging guidelines could therefore support more consistent logging practices and provide a foundation for improving the quality of generated log data.

Based on this motivation, this thesis investigates existing logging guidelines and practices and structures them into a catalogue according to relevant logging decisions.

1.2 Problem Statement

Automated log analysis techniques are widely used to help developers understand and maintain software systems. These techniques are commonly applied for tasks such as debugging, system monitoring, anomaly detection and root cause analysis. Most automated log analysis methods assume that log data are structured, consistent, and contain enough information to support analysis. In practice, this assumption often does not preserve this condition.

Log messages are usually written manually by developers, often without clear or standardised logging guidelines. As a result, log formats and message contents vary across software projects, development teams and programming languages. While some log messages contain detailed and structured information, others lack important context or follow inconsistent formats. This inconsistency makes it complicated for automated tools and reduces the reliability of log analysis. Although many existing studies focus on improving log analysis techniques, such as log parsing or anomaly detection, less attention is given to how log data is created in the first place. The quality of the input logs is often taken for granted. When log data are

inconsistent, incomplete or poorly structured, even advanced automated analysis techniques may fail to produce accurate and meaningful results.

Logging guidelines and logging practices have the potential to improve log quality by providing what should be logged, when logging should occur, how log messages should be structured. In practice, however, developers often rely on personal experience or make logging decisions on the spot without a systematic and reusable manner. Logging rules are rarely documented in a clear and reusable way and are often spread across documentation and source code. This makes it difficult to identify common practices or apply them consistently across different projects and programming languages.

As a result, the developers could not refer to a guideline or standard to keep logs consistent. Without a systematic approach to collecting and organising logging guidelines, opportunities to improve log quality and strengthen automated log analysis remain limited. Addressing this missing reference is important for increasing the reliability and effectiveness of automated log analysis in real-world software systems. Because of that a catalogue with logging guidelines is needed to improve log quality and to keep uniformed logs sustainable.

1.3 Goals and Research Questions

As there is no specific defined guidelines for logs, the goal is to create such foundation of logging guidelines into a catalogue to improve the log quality. This catalogue contains aspects of what to log, where to log, how to log and which log verbosity to use. This improves log statements at their source and make the resulting log data more suitable for automated log analysis.

To achieve this goal, we need to separate it into two aspects: first, collecting all existing guidelines from scientific literature and second, collecting all existing guidelines from open-source projects. Together we merge them into one big catalogue by classifying the guidelines, and sort and order it. Resulting into a catalogue with all guidelines needed for a good quality of log data.

Therefore, this thesis formulates the research questions, which are presented in Table 1.1, followed up with detailed research objectives in Table 1.2.

RQ1	What logging practices and challenges do software developers encounter in software development and maintenance?
RQ2	What logging guidelines are documented in literature and open-source software repositories?
RQ3	What themes and characteristics emerge from the identified logging guidelines?
RQ4	To what extent do the identified logging guidelines address the practices and challenges reported by software developers?

Table 1.1: Research Questions

RO1	Investigation of logging practices and challenges encountered by software developers
RO2	Collection of logging guidelines and practices from academic literature
RO3	Collection of logging guidelines and practices from open-source software repositories
RO4	Analysis and classification of logging guideline themes and characteristics
RO5	Evaluation of how well identified logging guidelines address developer practices and challenges

Table 1.2: Research Objectives

RQ1 investigates the logging practices and challenges encountered by software developers during software development and maintenance. Receive insights regarding logging practices and challenges show how software developers struggle to implement consistent logging statements (**RO1**).

Industry practices are investigated through a survey to collect from software developers. The survey focuses on log implementation, log analysis and fault localization practices. Understanding these practices and challenges from real world industry provides a reference for existing and missing logging guidelines.

RQ2 focuses on identify logging guidelines from literature and open-source repositories. In literature logging guidelines are verified through researchers and shows missing log quality(**RO2**).

But logging guidelines from open-source repositories are often implicit and not clearly documented. Therefore, different search queries and collection strategies are needed, because of different results in terms of relevance and coverage (**RO3**).

Covering academic literature and open-source repositories provides a balanced logging guideline from academical view and real world usage. By backtracing research papers this creates enough references for the academical perspective. Using different search queries covers the most open-source repositories. The collection aims to achieve sufficient coverage while maintaining relevance of the identified results. This resulting collection forms the empirical basis for developing a structured catalogue of logging guidelines from open-source software repositories and academical literature.

RQ3 examines the themes and characteristics that results from the identified logging guidelines. Classifying the collected guidelines according to their themes, pattern and characteristics supports the creation of a structured and reusable guideline catalogue (**RO4**). This includes aspects such as what information should be logged, where logging should occur, how logs should be structured and what log verbosity should be used.

Analysing different characteristics such as specifictiy, programming language and project type helps better to classify particular guidelines for particular language or project type. By structuring this guideline in this way the catalogue provides language, project specific guidelines and also common guidelines, which can apply for any language and software project.

RQ4 investigates the extent to which the identified logging guidelines address the practices and challenges mentioned by software developers. The findings from the developer study are compared with the logging guidelines identified and classified from open-source software repositories (**RO5**).

This comparison makes it possible to determine which developer practices and challenges are sufficiently addressed by existing guidelines. The analysis therefore connects real-world developer experiences with the recommendations found in software projects. Similar accordance can indicate guidelines that are particularly relevant to practical software development, while gaps may reveal logging challenges for which existing guidance is insufficient. This comparison contributes to assessing the practical relevance and coverage of the collected logging guidelines and helps identify opportunities for improving future logging guidance.

1.4 Organization of this Thesis

Following the introduction, Chapter 2, *Background*, explains the core understand about logging to understand subsequent chapters. Even related work is included here to understand current research.

Chapter 3, *Methodology*, explains how the research is investigated. Therefore, it will be explained in detail, how logging guidelines are collected for literature search and

open-source project. This includes search queries for the open-source and classification of both catalogues.

Chapter 5, *Results*, contains the classification and results of the catalogue. Hereby, detailed analysis is given, how each catalogue has evaluated.

Finally, *Conclusion* deals with the interpretation and usage of the catalogue. Moreover, it discusses problems that may occur for future.

2 Background

2.1 Software Logging 2.2 Logging Practices and Decisions 2.3 Automated Log Analysis 2.4 Existing Logging Guidelines 2.5 Mining Software Repositories 2.6 Related Work 2.7 Research Gap

The Background chapter of your thesis is an important part of your work, providing your readers with the necessary context and information to understand the problem you are addressing. To write a good Background chapter, it is important to remember that you should only describe the background that is necessary for your readers to know. You don't need to explain everything that you cover during the thesis's processing time.

One common mistake is to provide a lengthy description of official standards. This is not necessary as your readers can look them up themselves. Another common mistake is to provide extensive translations or transcriptions of official documents. It is usually enough to refer to the source material and provide a systematized overview of its content.

The key to writing a good Background chapter is its shortness. Keep in mind that this chapter should only provide the minimum information needed to understand your thesis's research question. This includes explaining any relevant background theories, prior research, and current state of the field. You should also describe any important terms or concepts that readers may not be familiar with.

A good way to approach the Background chapter is to think about it as an introduction to the problem you are addressing. This means providing enough context for readers to understand the importance of your research question and why it is worth investigating. Remember to keep your writing clear and concise, focusing on the key points that will help readers understand the background of your work.

2.1 Software Logging

2.2 Logging Guidelines and Practices

2.3 Automated Log Analysis

2.4 Logging in Open-Source Software

2.5 Related Work

Recommendations for logging exist in academic literature, industrial documentation, open-source projects and grey literature. However, these recommendations are distributed across many different sources and can vary considerably in their scope and level of detail. Some guidelines describe general logging principles, whereas others focus on particular technologies, programming languages or application domains. Furthermore, practical logging knowledge may be documented inside software repositories without being systematically collected or compared with recommendations from research.

Modern software systems generate large amounts of log data that collect information about their runtime behaviour. Log data are important for understanding how software systems operate and are used for tasks such as debugging, system monitoring, anomaly detection and root cause analysis. Inspecting log data from large scale and distributed systems is time consuming. As the volume of log data increases, automated processes for log analysis methods are introduced, including log parsing [9134790][8804456]. This approach transforms unstructured log messages into structured data that can be later used for automated log analysis.

The performance of a strong automated log analysis depends on the quality and consistency of the written log data. In practice, most logs are written without a clear logging guideline, which leads to inconsistent formats, different structures and results in a variety of logs, which is counterproductive for automation. The latest studies show that such inconsistencies can reduce accuracy and reliability [9793976][gholamian2021comprehensive][chen2021survey]. These challenges highlight the problem and motivate research on logging guidelines and practices to better support automated log analysis. Moreover, logging approaches to automate and improve logging practices have been examined [he2021survey] [7985651] and it shows there is still no approach, neither in industry [he2022empirical] or open source project [yuan2012characterizing], transform log messages into a guideline conform structure. It is also important to note that a considerable amount of 'grey' literature [medium_logging_best_practices_2022] [loggly_logging_scale_2022]

presents general logging practices and guidelines, which are often written by individual practitioners without a guarantee of academic or industry background.

List related work *and* the result of this work. What is the relevance of this work concerning your thesis? If necessary, *emphasize* some words in your text, for example words like *not* or *and* are sometimes crucial for understanding.

Tools: We do not make any strict guidelines regarding the tools that can be used to manage the related work. You can freely choose your own methodology and tools.

According to our experience, the following tools can be used as a starting point to find relevant papers:

- Search Engines: Google Scholar, Web of Science, ACM Digital Library, IEE-Explore, and *{Search Engine} + {Conference name} proceedings*
- Management Tools: Citavi, Zotero, Mendeley, and JabRef.

2.6 Research Gap

3 Methodology

This chapter describes

This chapter describes the setup for running the experiment with an overview of our experiment pipeline shown in Figure including data preparation, MIA feature and how we aggregate the feature.

This chapter introduces the importance of software logging and challenges of inconsistent logging practices and log data. Based on these challenges, the motivation and problem are described for this research. Afterwards, the next chapter describes goals and research questions about logging practices and guidelines. Followed up by the structure of this thesis.

3.1 Research Design

3.2 Developer Study

3.3 Literature Guideline Collection

1: - collect all from google scholar HowToLog Logging Guideline Logging Practice Logging Survey What To Log Where To Log from 2020 literature

- check literature references for more literature original literature

- extract important information check catalogue

3.4 Open-source Repository Guideline Collection

2: FULL LIST scraped list OPTList remove forks and same result for different queries OPTListwMetaData samefile including MetaData MetaList remove github removed repo , Date included , exclude/include + comments Prompts for search:

1. Broad High-Recall Query (Recommended Starting Point) Use this directly in GitHub search: ("logging guidelines" OR "logging best practices" OR "logging standards" OR "logging conventions" OR "logging policy") (path:/docs/ OR path:/github/

OR filename:README.md OR path:*/CONTRIBUTING.md) This will retrieve repositories containing explicit logging guidance documents. 2. Query Focused on “What to Log” ("what to log" OR "log levels" OR "when to log" OR "structured logging" OR "sensitive information") ("logging" OR "application logs") (path:/docs/ OR filename:README.md) Useful for extracting actionable recommendations such as: what events should be logged, log levels, sensitive data handling, tracing, observability. 3. Engineering Handbook / Internal Standards Query Many mature projects store logging guidance inside engineering standards rather than standalone logging docs. (logging observability tracing telemetry) ("engineering standards" OR handbook OR conventions OR practices OR guidelines) 4. Query for Structured Logging Practices ("structured logging" OR "correlation id" OR "trace id") ("best practices" OR guidelines OR standards) This is especially useful for distributed systems and microservices research. 5. Query for Security/Sensitive Logging Guidance (logging OR logs) ("sensitive data" OR PII OR GDPR OR credentials) ("guidelines" OR "best practices") This often reveals high-quality operational logging rules. 6. Query for Log Levels and Severity Policies ("log level" OR ERROR OR WARN OR INFO OR DEBUG) (logging guidelines OR logging conventions) This helps extract recommendations about: severity assignment, verbosity, debugging practices, production logging. 7. Query Targeting Large Mature OSS Projects You can restrict by stars: ("logging guidelines" OR "logging best practices") stars:>1000 Or by language/domain: ("logging guidelines") language:Java ("structured logging") language:Go 8. Query for Wiki/Markdown-Based Guidance Many repositories store guidelines in markdown files: (logging OR observability) extension:md (guidelines OR practices OR standards) 9. Query for Operational Observability Guidance Modern repositories may avoid the term “logging” and instead use: observability, telemetry, diagnostics, tracing. Use: (observability OR telemetry OR diagnostics OR tracing) (logging guidelines OR standards OR practices) 10. Query for Logging Anti-Patterns / Smells Very useful for building your catalogue of “avoid these” rules. ("logging smells" OR "bad logging practices" OR "logging anti-patterns")

- Get Query via QA - Scrap Data (only limited 5 page or search is not precise)
- Check Duplicated via different Query remove or merge and forks) - using cosinesimilarity removing 100- Remove not existing projects - get meta data and sort the most important one and recent most developers - extract information to catalogue

-include/exclude repos -resulted in almost 215 repos -filter not needed we use all -extracted what to log, where to log, how to log, log verbosity exclusive reference, project type, language, field LLM PROMPT:

"Give me all logging guidelines and practices related to what to log, where to log, how to log and log level verbosity from these repositories and also give me the projecttype specific in language and field:"

Also less devs repo interesting because of different and more guidelines (-internal categorization)

3.5 Guideline Classification

3.6 Comparison and Gap Analysis

3.7 Threats to Validity

4 Implementation

4.1 Duties and Agreements

To successfully write your thesis, you should definitely respect some rules. They are explained in the following sections.

4.1.1 Rules for Students

At the beginning of your thesis, estimate the complexity of the work you are going to have. Take into account that you will have problems with certain aspects of your thesis that will consume a lot of time. Consider times for recreation and delays you can not influence, for instance, asking your supervisor, waiting for orders to be shipped, complex problems during the implementation phase, and so on ...

For some students it is a good idea to agree upon a rough plan (with their supervisor) on how to make progress on their thesis and what goals to achieve. Milestones might help to control your progress. If you fail to meet a milestone in time, contact your supervisor on why this happened or when to expect it to be fulfilled.

If you have a problem, try to solve it on your own twice over. Some things just take time. In case you fail to solve it on your own, write an email to your supervisor and tell him/her about your problem and what you did to solve it. Make an appointment if necessary. Please do not jump right into his office, supervisors have other stuff to do, too.

For quotations, either use “quotation” or “quotation”. For some words, you should use a tilde to link them, for example, when referring to chapter 6 you should use it. Or use chapter 6. This prevents words from being separated by a line break or some other rare circumstances. Use BibTeX within your thesis and learn the different citation options [**Xie:2008:SBS**, **Newsome:05:DTA**].

One last piece of advice. Do not try to attend courses in parallel to your thesis. You should take this seriously and not think that writing a thesis is done quickly.

4.1.2 Rules for Supervisors

“With great power comes great responsibility” :-)

- It is very important, that if you want specific things to be done that you send these important instructions by mail. Your student might be in a moment of confusion when telling him/her.
- Attend the “Colloquium” and give your student the feeling, that this is important to you, too.
- Offer your students the opportunity to talk to you right after the “Colloquium”. While discussing things, tell your student to write down the results of this discussion and tell him/her to send you this summary by mail to ensure (if necessary), you did not talk at cross purposes.
- Last but not least: Please be gentle to your students :-)

4.2 Hints on Typesetting

To get this template running, we recommend using VSCode and Remote Containers.

4.2.1 Structuring Text, English Hints

Another text about the following sections ...

4.2.1.1 Text

Always try to structure your text in a manner that makes sense. Either use indentations, itemize or enumeration environments.

This sentence will have an indentation at the beginning. Now an enumeration starts:

1. One.
2. Two.
3. Three.

Sometimes you do not want an indentation. Use the `noindent` command in such a case.

One Is the first number.

Two Is the second number.

Glossary Use the glossary package for acronyms. In addition, the glossay package can help you to avoid typing the same word in different ways. For example students tend to mix-up the writing of the word *User-Agent* in different ways: user-agent, User-Agent, user agent, User agent. This inconsistency can be avoided by just using the glossary entry: User-Agent (UA).

4.2.1.2 English Hints

- Use an active voice and avoid using passive wherever possible.
- *Always* use the present tense (especially when you refer to content that occurs later in your text). For example:
 - *wrong*: The next chapter *will* explain ...
 - *correct*: The next chapter explains ...
- Either use American English or British English, but do not mix (e.g. summarize vs. summarise, analyze vs. analyse, ...). American English is preferred.
- Do not use filler words.
 - omit: “some kind of” and others ...
- Never use a comma before “that”.
- For enumerations, always use a comma before “and”: “...module 1, module 2, and module 3.”.
- The title of your thesis is capitalized except for words like and, or, with, the, a ...
- *Always* address the reader using the third person: “As one can see from ...” and not “As you can see ...”.
- All tables, figures have to be explained very briefly in the text itself.
- Always use correct quantifications:
 - *wrong*: ... a small amount of runs ...
 - *correct*: ... at most three runs ...
- Never use “I”. Depersonalize your sentences or use “we” if necessary.

- Read *The Elements of Style* by William Strunk, Jr., which is for example available at <http://www.crockford.com/wrrrld/style.html>. The (short) book provides an overview of typical errors and helps you to significantly improve your English.

4.2.1.3 General Hints

- Use non-breaking small space for some abbreviation
 - z. B.
 - u. a.
 - e. g.
- Use a non-breaking space just before references, parentheses and so which shall not begin at the beginning of a new line. This sentence will not break here (and here).
- Did you notice the overfull horizontal box (hbox)? You should avoid these! Underfull boxes are not that bad. But only fix them when most of the section, paragraph etc is ready. Otherwise you have to fix them more than once. You can tell L^AT_EX when to break a word if it does not do it correctly. Just put a \- at the corresponding position in the word. Vertical overfull boxes (vbox) occur if the document uses \flushbottom instead of \raggedbottom. That way, L^AT_EX ensures that each page ends with the last sentence in the last line (except for the final line in a section). To enforce this, L^AT_EX sometimes has to add extra vertical space between, e. g., paragraphs. Overfull vertical boxes are hard to fix, as additional content needs to be added or even has to be removed sometimes. Keep in mind that any changes to the type area (Satzspiegel) might produce many additional over- or underfull boxes (and of course it will fix other boxes).
- Read <ftp://ftp.dante.de/tex-archive/info/german/l2tabu/l2tabu.pdf>. Really, read it.
- You can find many more good information at <http://www.dante.de/CTAN/info/lshort/german/l2kurz.pdf>
- The KomaScript guide is very useful: <ftp://ftp.dante.de/pub/tex/macros/latex/contrib/koma-script/scrguide.pdf>

4.2.2 Formulas, Figures, Tables, Definitions

4.2.2.1 Formulas

Using the method shown in Table XX for all three functions yields.

$$f_a^4 = 0x2C79 = abc + ac + ad + bc + a + b + d + 1 \quad (4.1)$$

$$f_b^4 = 0x6671 = abd + acd + bcd + ab + ac + bc + a + b + d + 1 \quad (4.2)$$

$$f_c^5 = 0x7907287B = cde + abde + ade + de + abce + bce + ce + be + bcd \\ + acd + bd + d + bc + ab + b + 1 \quad (4.3)$$

When typesetting formulas, pay special notice on constants, variables, and units:

$$\mathcal{F}_\omega\{x(t)\} = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt \quad (\text{Fourier-Transformation})$$

The use of constants, variables and units is explained by “Rohde & Schwarz” in their famous document “Der korrekte Umgang mit Größen, Einheiten und Gleichungen” [**msc:correctuse**]. These rules are in compliance with ISO-31. Consequently, always typeset the following in italics:

- Variables like k , x , ...
- Functions like $f(x)$, ...
- Physical constants like c_0 , ...
- Indices that are variables or physical units, like $a_{i,j}$ or c_V .

4.2.2.2 Fontawesome

In case you need some extra symbols, we recommend using Fontawesome , , .

4.2.2.3 Figures

Figures and tables are important to explain things. Here are some rules that apply, when using figures:

- Whenever possible use vector graphics (eps, pdf, svg, ...) instead of bitmap graphics (jpg, gif, ...).

- All figures should have the same font and size (do not scale them or the size will change) and “style” (line strength, arrow heads, ...).
- Some employees of the chair need all figures in `.eps`. However, do *not* convert your `.jpg` and `.png` to `.eps`, instead use a *wrapper* program to wrap these file types into the `.eps` format. As a consequence, you are forced to use `latex` to typeset your document instead of `pdflatex`. Appropriate wrapper programs can be found here:
 - Windows: click
 - Linux/Mac: click
- **Always** try to use your own figures, so you do not run into copyright problems and it is easier for us, to reuse these figures for papers. You might want to have a look at these tools to create your own figures:
 - Windows: MS Visio (available via MSDNAA), Graphviz, Gnuplot ...
 - Linux: xfig/jfig, IPE, Graphviz, Gnuplot ...
 - Mac: IPE, Graphviz, Gnuplot, OmniGraffle (commercial, academic licensing available) ...

There are many possibilities on how to include figures, here is just one example on how to do it. In case you need further assistance, please google for `l2picfaq`.

4.2.2.4 Tables

There are many possibilities on how to create and include tables. From a typographic point of view, *one should avoid any vertical lines*, cf. Table 4.1.

Table 4.1: Captions for tables are *always above* the table and give a short but informative description of the table. Always use full sentences here and end them with a full stop.

Amount ^a	Price	Component	
		Description	Role
23	1.234 \$	good stuff	important
multirow example the other row	x	y	XXX
42	43.123,13 ^b	good stuff	important

^aThis is a footnote inside a table, you need a minipage for this to work.

^bThis is another footnote inside a table.

4.2.3 Glossaries

The `glossaries` package in LaTeX allows you to create glossaries and lists of acronyms in your document. This is particularly useful in academic writing, where it is important to ensure consistency in the use of terminology.

To create an acronym using `glossaries`, you first need to define it using the `newacronym` command. For example, `\newacronym{ua}{UA}{User-Agent}` defines the acronym “UA” with the corresponding expansion “User-Agent”.

To create a glossary entry, you can use the `newglossaryentry` command. For example, `\newglossaryentry{xss}{name={XSS}, description={Cross-Site-Scripting}, ...}` creates a glossary entry with the label “xss”, the name “XSS”, and the description “Cross-Site-Scripting...”.

To reference an acronym or glossary entry in your text, you can use the `gls` command. For example, `The \gls{sso} system enables...` will produce “The Single Sign-On (SSO) system enables...”.

Glossaries are important in academic writing because they help ensure consistency in the use of terminology. By defining acronyms and glossary entries up front, students can ensure they are using the correct terminology throughout their thesis. This helps to prevent variations such as “Single Sign-On”, “Single-Sign-On”, or “Single-Sign On”, which can be confusing for readers and detract from the clarity of the writing.

4.2.4 Listings

4.2.4.1 Listings

We recommend using the `minted` package for code highlighting.

```
1 def myFunction(param):  
2     return param + 1  
3  
4 myFunction(5)
```

You should thoroughly document your code using comments and (best case) by using a documentation system like Doxygen. Please ask your supervisor for additional rules (e.g. which repository system to use, etc.). Regularly commit your changes and backup your data!

5 Results

It is **current best practices** to provide a brief overview on your most interesting results in the beginning.

	Browser	XSS	SQLi	CSRF	Clickjacking
	Firefox				
	Chrome				
	Safari				

Table 5.1: Write a short message that everyone should understand, e.g., “All browsers were found vulnerable (●) to Clickjacking” instead of simply writing “Evaluation results.”. Also describe what the symbols mean.

6 Conclusion

Writing a good conclusion is an essential part of any academic work. It is your final chance to leave a lasting impression on the reader and make sure that your work is remembered for its contribution to the field. While many students may be tempted to simply summarize their results in the conclusion, this approach is often too simplistic and fails to provide the reader with any real insight or understanding of the significance of your work.

To write a good conclusion, it is important to not only summarize your results but also to explain what the reader can learn from your thesis. This means highlighting the key insights and contributions that your work has made to the field. What have you discovered that is new and important? What have you contributed to the existing body of knowledge? How does your work relate to other studies in the field?

In addition to highlighting your contributions, a good conclusion should also identify areas for future work. What questions remain unanswered? What new avenues of research have been opened up by your work? What are the key challenges that need to be addressed in order to make further progress in the field?

By providing these insights and identifying future directions for research, your conclusion will not only leave a lasting impression on the reader but also provide a valuable resource for future scholars looking to build on your work. Remember, a good conclusion is not just a summary of your results but a thoughtful reflection on the significance of your work and its contribution to the field.

List of Figures

List of Tables

1.1	Research Questions	4
1.2	Research Objectives	4
4.1	This is the short caption for the <i>List of Tables</i>	20
5.1	Write a short message that everyone should understand, e.g., “All browsers were found vulnerable (●) to Clickjacking” instead of simply writing “Evaluation results.”. Also describe what the symbols mean.	23

Bibliography

- [1] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. “Logram: Efficient Log Parsing Using nn -Gram Dictionaries”. In: *IEEE Transactions on Software Engineering* 48.3 (2022), pp. 879–892. DOI: 10.1109/TSE.2020.3007554.
- [2] Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R. Lyu. “Tools and Benchmarks for Automated Log Parsing”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 2019, pp. 121–130. DOI: 10.1109/ICSE-SEIP.2019.00021.
- [3] Zanis Ali Khan, Donghwan Shin, Domenico Bianculli, and Lionel Briand. “Guidelines for Assessing the Accuracy of Log Message Template Identification Techniques”. In: *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 2022, pp. 1095–1106. DOI: 10.1145/3510003.3510101.
- [4] Sina Gholamian and Paul A. S. Ward. *A Comprehensive Survey of Logging in Software: From Logging Statements Automation to Log Mining and Analysis*. 2022. arXiv: 2110.12489v2 [cs.SE]. URL: <https://arxiv.org/abs/2110.12489>.
- [5] Boyuan Chen and Zhen Ming (Jack) Jiang. “A Survey of Software Log Instrumentation”. In: *ACM Comput. Surv.* 54.4 (May 2021). ISSN: 0360-0300. DOI: 10.1145/3448976. URL: <https://doi.org/10.1145/3448976>.
- [6] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. “A Survey on Automated Log Analysis for Reliability Engineering”. In: *ACM Comput. Surv.* 54.6 (July 2021). ISSN: 0360-0300. DOI: 10.1145/3460345. URL: <https://doi.org/10.1145/3460345>.
- [7] Boyuan Chen and Zhen Ming Jiang. “Characterizing and Detecting Anti-Patterns in the Logging Code”. In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 2017, pp. 71–81. DOI: 10.1109/ICSE.2017.15.

- [8] Shilin He, Xu Zhang, Pinjia He, Yong Xu, Liqun Li, Yu Kang, Minghua Ma, Yining Wei, Yingnong Dang, Saravanakumar Rajmohan, and Qingwei Lin. “An Empirical Study of Log Analysis at Microsoft”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2022. Singapore, Singapore: Association for Computing Machinery, 2022, pp. 1465–1476. ISBN: 9781450394130. DOI: 10.1145/3540250.3558963. URL: <https://doi.org/10.1145/3540250.3558963>.
- [9] Ding Yuan, Soyeon Park, and Yuanyuan Zhou. “Characterizing Logging Practices in Open-Source Software”. In: *2012 34th International Conference on Software Engineering (ICSE)*. 2012, pp. 102–112. DOI: 10.1109/ICSE.2012.6227202.
- [10] Medium. *13 Best Practices to Improve Your Logging*. <https://medium.com/cogito-engineering/13-best-practices-to-improve-your-logging-75665183f05b>. Accessed: 2022-05-18. 2022.
- [11] Loggly. *30 Best Practices for Logging at Scale*. <https://www.loggly.com/blog/30-best-practices-logging-scale/>. Accessed: 2022-05-18. 2022.

A Java Code